

J.A.R.V.I.S. — Phase 2 (v3)

Software Requirements Specification | Cognitive Architecture Edition

Phase 1 Planning	Phase 2 Requirements	Phase 3 Design	Phase 4 Implementation	Phase 5 Testing	Phase 6 Deployment
------------------	----------------------	----------------	------------------------	-----------------	--------------------

1. Functional Requirements — Core System

ID	Module	Requirement	Priority
FR-01	Voice Input	Wake word detection (Porcupine offline)	High
FR-02	Voice Input	Speech-to-text via Whisper (local, webrtcvad for VAD)	High
FR-03	Voice Output	Dual voice: ElevenLabs (online) + pyttsx3 fallback (offline)	High
FR-04	PC Control	Open/close apps via psutil + os.startfile (whitelist enforced)	High
FR-05	PC Control	Web search + browser open (DuckDuckGo, no API key)	High
FR-06	PC Control	Read/write files sandboxed to home directory	High
FR-07	Email	Compose + send via Gmail SMTP with voice confirmation gate	Medium
FR-08	Coding	Voice Q&A; + file review via LLM (qa/review modes)	High
FR-09	Coding	Read file, suggest fixes, run tests via subprocess	Medium
FR-10	Trading	Live prices: NYSE, crypto, Forex, NGX via yfinance + Alpha Vantage	High
FR-11	Trading	Market news via Alpha Vantage NEWS_SENTIMENT endpoint	Medium
FR-12	Trading	Technical analysis (RSI, MA) — DEFERRED to v1.1	Low

FR-13	Memory	Short-term: deque(20) conversation buffer per session	High
FR-14	Memory	Long-term: Mem0 unified API (semantic + knowledge graph)	High
FR-15	LLM	Claude API tool_use routing through core/llm.py only	High

2. Functional Requirements — Cognitive Architecture

Critical architecture note — what 'learning' means: The LLM (Claude, GPT-4o, Mistral) never changes. Its weights are frozen and trained by its provider. What improves over time is the *context injected into every API call* — richer notes, better-organised facts, deeper domain knowledge retrieved from the memory systems below. This pattern is called RAG (Retrieval Augmented Generation). Every major AI product (Perplexity, Cursor, GitHub Copilot, NotebookLM) uses it. The LLM is the cortex — general intelligence, frozen. The memory architecture is the hippocampus — experience, accumulated over time.

ID	System	Requirement	Priority
FR-COG-01	Episodic memory	System shall log every agent action to SQLite episodic store with timestamp, agent_id, task_id, action_type, inputs, outputs, and success_score	High
FR-COG-02	Memory consolidation	System shall run a nightly consolidation job that extracts durable facts from episodic logs, stores them in semantic memory, and applies forgetting curve decay	High
FR-COG-03	Forgetting curve	All memory entries shall have a float activation_score (0.0–1.0) that decays 5% daily and spikes +0.3 on retrieval. Entries below 0.1 are archived	High
FR-COG-04	Knowledge graph	System shall maintain a knowledge graph (NetworkX → Neo4j) storing entity relationships. Every agent action automatically writes edges to the graph	High
FR-COG-05	Spreading activation	Memory retrieval shall return not just the queried node but all nodes within 2 hops in the knowledge graph, enabling associative recall	High
FR-COG-06	Procedural memory	After 5+ similar completed tasks, the consolidation job shall extract a reusable skill pattern and store it in the skill library for future prompt injection	High
FR-COG-07	Confidence scoring	Agents shall tag retrieved memories with their confidence_score before acting. Score < 0.5 triggers autonomous research before task execution	High

FR-CO G-08	Metacognitive map	Each agent shall maintain a JSON competence map of topics with coverage scores, updated after every completed task	High
FR-CO G-09	Context budget	The build_context() function shall enforce a token budget per context section, compressing by LLM summarisation when a section overflows	High
FR-CO G-10	Mem0 unified API	The memory layer shall use Mem0 as the unified API, wrapping ChromaDB and knowledge graph. Episodic, procedural, confidence, and metacognitive modules are built on top	High

3. Non-Functional Requirements

ID	Category	Requirement
NFR-01	Performance	Wake word detection latency < 500ms
NFR-02	Performance	End-to-end response (voice in → voice out) < 5 seconds
NFR-03	Privacy	Raw audio never stored to disk. Only transcribed text sent to cloud APIs
NFR-04	Privacy	Only text (not audio) sent to LLM APIs
NFR-05	Reliability	API failures handled with exponential backoff (1s, 2s, 4s) + graceful fallback
NFR-06	Usability	System starts with single command: python jarvis.py
NFR-07	Security	All API keys in .env only. Never hardcoded. Never in config.json
NFR-08	Maintainability	Each module independently testable. Test stub required per implementation step
NFR-09	Platform	Windows 10/11, Python 3.11+
NFR-CO G-01	Memory perf	Semantic + graph retrieval combined < 300ms per LLM call
NFR-CO G-02	Consolidation	Nightly consolidation completes within 30 min for 10,000 episodic events
NFR-CO G-03	Graph scale	Knowledge graph supports 100k nodes / 1M edges before Neo4j migration
NFR-CO G-04	Context	No LLM call exceeds 180,000 tokens (20k safety margin preserved)
NFR-CO G-05	Background	Forgetting curve decay runs as low-priority background task — never blocks voice

4. External Integrations (Complete List)

Integration	Purpose	Free?
Picovoice Porcupine	Wake word detection (offline)	Yes
OpenAI Whisper	Speech-to-text (local model)	Yes — local
webrtcvad	Voice activity detection	Yes — local

Anthropic Claude API	LLM core brain + tool_use	Limited credits
ElevenLabs	Realistic TTS voice (online)	Yes — free tier
pyttsx3	Offline TTS fallback	Yes
mem0ai	Unified memory API (semantic + knowledge graph)	Yes — open source
chromadb	Vector store backing Mem0	Yes — local
networkx	Knowledge graph — Phase 1	Yes
neo4j (v2)	Knowledge graph at scale	Free Community
sentence-transformers	Local text embeddings	Yes — local
yfinance	Market prices (primary)	Yes
alpha-vantage	Market prices (fallback) + news	Yes — free tier
duckduckgo-search	Web search (no API key)	Yes
smtplib / Gmail SMTP	Email sending	Yes
pyautogui	Screen capture + mouse/keyboard automation	Yes
psutil	Process management (close apps)	Yes
colorama	Windows ANSI terminal colours	Yes
plyer	Windows toast notifications	Yes
python-dotenv	Secret management (.env)	Yes
tiktoken	Token counting for context budget	Yes
APScheduler	Background task scheduler (nightly consolidation)	Yes
beautifulsoup4	HTML parsing for web research (v2)	Yes
pypdf	PDF text extraction for research (v2)	Yes
youtube-transcript-api	YouTube captions — no video download (v2)	Yes
pytest + pytest-mock	Unit testing framework	Yes

Trading safety note: FR-12 (RSI, moving averages) is DEFERRED to v1.1. No auto-trade execution in v1 or v2. Market data is read-only. The ethical hacker agent (v2) operates only on systems listed in config.json authorized_targets — hardcoded guardrail, not overrideable by prompt.